

1.实验目的

- 1.掌握汇编语言中 DOS 系统功能调用（int 21h 等）的使用方法，理解系统调用在字符输入、输出及串处理中的作用。
- 2.熟悉并能够正确运用串操作指令（如 `movsb`、`movsw`、`cmps`、`scasd` 等），实现对字符串的搜索、比较及替换操作。
- 3.综合运用前几次实验中学习的汇编指令、循环结构、地址计算与 DEBUG 调试技巧，完成对字符串的查找与替换等功能，实现小型字符串处理程序。

2.实验环境

- 操作系统：Windows（Windows 11）
- 运行环境：DOSBox 模拟器（模拟 x86 实模式）
- 实验与调试工具：
 - DEBUG（DEBUG.exe，用于调试 16 位汇编 COM 程序）
 - 需要时可使用 MASM、LINK 编译汇编程序

3.实验内容

- 1.从键盘输入两个字符串：
 - A 串，例如 `"software"`
 - B 串，例如 `"school of software engineering"`
- 2.判断 A 是否在 B 中存在。
 - 若存在，输出 `"FOUND"`
 - 若不存在，输出 `"NOT FOUND"`
 - 需要使用串比较相关指令（如 `cmps` 或循环比较）
- 3.实现字符串替换：
 - 在确认 B 串中存在 A 串的情况下，使用第三个字符串 C 去替换 B 中第一次出现的 A。
 - C 的长度可以与 A 不同，需要正确处理长度变化。
 - 替换后的 B 串需要在屏幕上显示出来。

4.实验具体实现

代码

```
1  data segment
2      bufferA db 32
3              db 0
4              db 32 dup('$')
5      db 4 dup(0)
6      bufferB db 32
7              db 0
8              db 32 dup('$')
9      db 4 dup(0)
10     bufferC db 32
11             db 0
12             db 32 dup('$')
13     db 4 dup(0)
14
15     temptail db 80 dup(0)
16
17     promptstringA db 'input a string:',0AH,0DH,'$'
18     promptstringB db 'input b string:',0AH,0DH,'$'
19     promptstringC db 'input c string:',0AH,0DH,'$'
20     promptstringENTER db 0AH,0DH,'$'
21     foundstring db 'found & replaced:',0AH,0DH,'$'
22     notfoundstring db 'not found it',0AH,0DH,'$'
23
24 data ends
25
26 code segment
27     assume cs:code,ds:data,es:data
28
29     start:
30         mov dx,data
31         mov ds,dx
32         mov es,dx
33
34         mov dx,offset promptstringA
35         mov ah,09H
36         int 21H
37
38         mov dx,offset bufferA
39         mov ah,0AH
40         int 21H
41
```

```
42     mov dx,offset promptstringENTER
43     mov ah,09H
44     int 21H
45
46     mov dx,offset promptstringB
47     mov ah,09H
48     int 21H
49
50     mov dx,offset bufferB
51     mov ah,0AH
52     int 21H
53
54     mov dx,offset promptstringENTER
55     mov ah,09H
56     int 21H
57
58     mov dx,offset promptstringC
59     mov ah,09H
60     int 21H
61
62     mov dx,offset bufferC
63     mov ah,0AH
64     int 21H
65
66     mov si,offset bufferC
67     add si,2
68     mov cl,[si-1]
69     mov ch,0H
70     add si,cx
71     mov byte ptr [si],'$'
72
73     mov dx,offset promptstringENTER
74     mov ah,09H
75     int 21H
76
77     mov al,[bufferA+1]
78     mov ah,00H
79     mov cx,ax
80     mov bp,cx
81
82     mov bl,[bufferB+1]
83     mov bh,00H
84     mov bx,bx
85
```

```
86     cmp bx,cx
87     jb notfound
88     sub bx,cx
89
90     mov dx,00H
91
92     again:
93         cmp dx,bx
94         jg notfound
95
96         mov si,offset bufferB+2
97         add si,dx
98
99         mov di,offset bufferA+2
100        mov cx,bp
101
102        cld
103        push si
104        push di
105        push cx
106        repe cmpsb
107
108        pop cx
109        pop di
110        pop si
111
112        jnz compare_fail
113
114        jmp found
115
116    compare_fail:
117        inc dx
118        jmp again
119
120    found:
121        mov di,offset temptail
122        xor ax,ax
123        mov cx,80H
124        cld
125        rep stosb
126
127        mov di,offset temptail
128        mov cx,dx
129        cmp cx,0H
```

```
130     je copy_c
131
132     mov si,offset bufferB+2
133     cld
134     rep movsb
135
136 copy_c:
137     mov cl,[bufferC+1]
138     mov ch,0H
139     cmp cx,0H
140     je copy_suffix
141
142     mov si,offset bufferC+2
143     cld
144     rep movsb
145
146 copy_suffix:
147     mov ax,dx
148     mov cl,[bufferA+1]
149     mov ch,0H
150     add ax,cx
151     mov bp,ax
152
153     mov cl,[bufferB+1]
154     mov ch,0H
155     sub cx,bp
156
157     cmp cx,0H
158     jle finish_replace
159
160     mov si,offset bufferB+2
161     add si,bp
162
163     cld
164     rep movsb
165
166 finish_replace:
167     mov al,[bufferB+1]
168     mov ah,0H
169     mov cl,[bufferA+1]
170     mov ch,0H
171     sub ax,cx
172     mov cl,[bufferC+1]
173     mov ch,0H
```

```
174     add ax,cx
175     mov [bufferB+1],al
176
177     mov cl,al
178     mov ch,0H
179     mov si,offset temptail
180     mov di,offset bufferB+2
181     cld
182     rep movsb
183
184     mov dx,offset foundstring
185     mov ah,09H
186     int 21H
187
188     mov si,offset bufferB+2
189     add si,ax
190     mov byte ptr [si],'$'
191
192     mov dx,offset bufferB+2
193     mov ah,09H
194     int 21H
195
196     jmp endpro
197
198 notfound:
199     mov dx,offset notfoundstring
200     mov ah,09H
201     int 21H
202
203 endpro:
204     mov ax,4C00H
205     int 21H
206
207 code ends
208 end start
```

编译结果

```
D:\>ml lab6.asm
Microsoft (R) Macro Assembler Version 6.11
Copyright (C) Microsoft Corp 1981-1993. All rights reserved.

Assembling: lab6.asm

Microsoft (R) Segmented Executable Linker Version 5.60.339 Dec 5 1994
Copyright (C) Microsoft Corp 1984-1993. All rights reserved.

LINK : warning L4017: /r : unrecognized option name; option ignored
Object Modules [.obj]: /r lab6.obj
Run File [lab6.exe]: "lab6.exe"
List File [nul.map]: NUL
Libraries [.lib]:
Definitions File [nul.def]:
LINK : warning L4021: no stack segment

D:\>
```

运行结果

```
D:\>lab6
input a string:
aaa
input b string:
bbbbbaaaddd
input c string:
cccc
found & replaced:
bbbbbccccddd
D:\>
```

5. 实验分析与总结

本次实验旨在掌握汇编语言中 **DOS 系统功能调用 (int 21h)** 的使用方法，特别是用于字符输入输出（如 **09H** 和 **0AH** 功能）的作用，并熟练运用 **串操作指令**（如 **cmps** 和 **movsb**）来实现字符串的查找和替换功能。

核心功能实现与分析

1. 字符串输入与显示:

- 程序使用了 **09H 功能** 显示字符串，输出提示信息。
- 使用了 **0AH 功能** 从键盘输入字符串，获取 A、B、C 三个字符串。该功能将输入的最大长度存放在缓冲区第一个字节，实际输入长度存放在第二个字节，字符串从第三个字节开始。
- 在接收 C 串输入后，代码通过计算实际长度并在其末尾添加 **\$** 终止符，使其可以直接使用 **09H 功能** 进行输出。

2. 字符串查找（A 串在 B 串中）：

- 首先，程序计算了 A 串和 B 串的实际长度，并检查 B 串的长度是否大于 A 串的长度。
- 查找使用了嵌套循环结构：
 - 外层循环（**again**）负责移动 B 串的起始比较位置。
 - 内层循环使用了 **repe cmpsb** 指令进行字符比较。在 CX 计数不为零且 ZF=1（字符相等）时重复比较 DS:SI 和 ES:DI 指向的字节。
 - 查找成功（ZF=1）则跳转到 **found**；失败则增加 DX（偏移量）并继续外层循环。

3. 字符串替换:

- 替换的逻辑是：**前缀 → C 串 → 后缀**。
- 替换时，先将 B 串中 A 串首次出现位置之前的部分（即前缀）复制到临时缓冲区 **temptail** 中，DX 寄存器保存了 A 串在 B 串中首次出现的起始偏移量。
- 接着，将 C 串复制到 **temptail** 中前缀的末尾。
- 然后，计算 B 串中 A 串之后的部分（即后缀）的起始位置和长度，并将其复制到 **temptail** 中 C 串的末尾。
- 最后，更新 B 串的实际长度，并将 **temptail** 中的结果复制回 **bufferB** 中。

调试与优化

实验代码中正确地使用了 **cld** 清除方向标志位，确保 **cmps** 和 **movsb** 等串操作指令按地址递增方向进行。在查找比较时，为保护寄存器现场，正确地使用了 **push** 和 **pop** 操作。

本次实验的一个主要难点在于处理替换时 **字符串长度的变化**。由于 C 串和 A 串的长度可能不同，需要精确计算替换后 B 串的总长度，并在 `temptail` 缓冲区中进行拼接。代码中通过 AX 寄存器完成了这一复杂的长度计算，确保了替换后的 B 串能正确显示。

总结

通过本次实验，我不仅巩固了 **DOS 系统调用** 在控制台 I/O 中的应用，也深刻理解了 `cmps` 和 `movsb` 等串操作指令在汇编语言中处理字符串的效率和精确性。特别是字符串替换部分，要求对内存地址、长度计算以及 SI/DI 等寄存器的使用有精确的控制，极大地提升了我对汇编程序设计和底层数据操作的理解能力。